

Algorithmique et programmation impérative

Chapitre 3 : Fonctions

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 3. Fonctions

- 1 Appels de fonctions
- 2 Définir de nouvelles fonctions
- 3 Effets de bord
- 4 Flot d'exécution
- 5 Documenter son code
- 6 Un petit mot sur les Tuples (parenthèse)
- 7 Erreurs fréquentes
- 8 Exercices interactifs
- 9 Glossaire

Appels de fonctions

Faire appel à une fonction

Définition

Une **fonction** est une séquence d'instructions à laquelle on donne un nom. Elle peut prendre en entrée une liste d'**arguments** et fournit en sortie une **valeur de retour**.

On peut voir une fonction comme une **boîte noire** qui effectue un travail.

- les arguments sont ce qu'elle a besoin pour faire son travail (entrées)
- la valeur de retour est le résultat de son travail

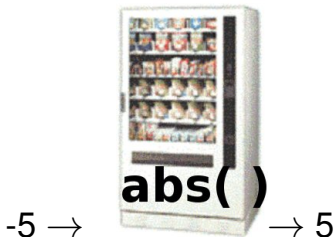


Faire appel à une fonction

- on reconnaît l'appel à une fonction par l'utilisation de parenthèses qui entourent les arguments
- on dit qu'une fonction "prend" quelque chose comme argument et "retourne" un résultat

Exemple : la fonction `abs` prend un nombre en entrée et retourne sa valeur absolue.

```
>>> x = -5
>>> y = abs(x)
>>> print(y)
5
>>> print(abs(8.0))
8.0
```



Faire appel à une fonction

Exemple : la fonction `print` affiche les valeurs de ses arguments (elle n'a pas pour but de retourner quelque chose)

Exemple : la fonction `int` convertit une chaîne en entier

```
>>> int('32')
```

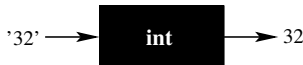
```
32
```

```
>>> s = '100' + '20'
```

```
>>> x = int(s)
```

```
>>> print(x)
```

```
10020
```



Remarque : `print` est une fonction en Python 3 (parenthèses nécessaires) mais en Python 2, c'est une instruction (pas de parenthèses nécessaires).

Fonctions de conversion de types

Python fournit une série de fonctions permettant de convertir le type d'une valeur.

- `int()` : chaîne de caractères → entier : ok si la chaîne représente un entier

```
>>> int('32')
```

```
32
```

```
>>> int('Bonjour')
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
ValueError: invalid literal for int() with base 10: 'Bonjour'
```

- `int()` : réel → entier : ne garde que la partie entière

```
>>> int(3.99999)
```

```
3
```

```
>>> int(-2.3)
```

```
-2
```

Fonctions de conversion de types

- `float()` : entier ou chaîne $\rightarrow$ réel : ok si la chaîne représente un réel

```
>>> float(32)
```

```
32.0
```

```
>>> float('3.14159')
```

```
3.14159
```

- `str()` : entier ou réel $\rightarrow$ chaîne

```
>>> str(32)
```

```
'32'
```

```
>>> str(3.14159)
```

```
'3.14159'
```


Modules

Définition

Un **module** est un fichier qui contient une collection de fonctions et variables reliées (ainsi que d'autres choses, voir plus tard). Il faut l'importer pour l'utiliser.

Pour accéder aux fonctions (**appels** de fonctions) ou aux variables définies dans un module, on utilise la **notation "point"** :

```
>>> import math
>>> x = math.sqrt(9)
>>> print(x)
```

3.0

```
>>> print(math.pi)
```

3.141592653589793

Modules et fonctions

Pour éviter de devoir donner le nom du module lors d'un appel à une fonction, on peut importer nommément une fonction.

```
>>> from math import sqrt
>>> sqrt(16)
4.0
```

On peut aussi importer de cette manière toutes les fonctions d'un module.

```
>>> from math import *
>>> sin(pi/2) + cos(pi/2)
1.0
```

Modules et fonctions

Si vous importez tout un module via *, faites attention aux conflits si des fonctions (ou des variables) ont le même nom. La dernière importée ou définie l'emporte !

```
>>> e = 4
>>> f = 5
>>> g = 6
>>> print(e, f, g)
4 5 6
>>> from math import *
>>> print(e, f, g)
2.718281828459045 5 6
```

Documentation sur les modules

Comment connaître les fonctions d'un module ?

Option 1 : Consultez <https://docs.python.org/3/library>

Python » 3.5.2 » Documentation » The Python Standard Library » 9. Numeric and Mathematical Modules » | [previous](#) | [next](#) | [modules](#) | [index](#)

Table Of Contents

- 9.2. **math** — Mathematical functions
 - 9.2.1. Number-theoretic and representation functions
 - 9.2.2. Power and logarithmic functions
 - 9.2.3. Trigonometric functions
 - 9.2.4. Angular conversion
 - 9.2.5. Hyperbolic functions
 - 9.2.6. Special functions
 - 9.2.7. Constants
- Previous topic
 - 9.1. **numbers** — Numeric abstract base classes
- Next topic
 - 9.3. **cmath** — Mathematical functions for complex numbers
- This Page
 - [Report a Bug](#)
 - [Show Source](#)

9.2. **math** — Mathematical functions

This module is always available. It provides access to the **mathematical** functions defined by the C standard.

These functions cannot be used with complex numbers; use the functions of the same name from the **cmath** module if you require support for complex numbers. The distinction between functions which support complex numbers and those which don't is made since most users do not want to learn quite as much **mathematics** as required to understand complex numbers. Receiving an exception instead of a complex result allows earlier detection of the unexpected complex number used as a parameter, so that the programmer can determine how and why it was generated in the first place.

The following functions are provided by this module. Except when explicitly noted otherwise, all return values are floats.

9.2.1. Number-theoretic and representation functions ¶

math.ceil(x)
Return the ceiling of *x*, the smallest integer greater than or equal to *x*. If *x* is not a float, delegates to `x.ceil()`, which should return an *Integral* value.

math.copysign(x, y)
Return a float with the magnitude (absolute value) of *x* but the sign of *y*. On platforms that support signed zeros, `copysign(1.0, -0.0)` returns `-1.0`.

math.fabs(x)
Return the absolute value of *x*.

math.factorial(x)

Documentation sur les modules

Option 2 : Utilisez `help()` et `dir()` dans la console interactive

```
>>> import math
>>> help(math)
(longue aide: voir console interactive)

>>> dir(math)

['__doc__', '__file__', '__loader__', '__name__', '__package__',
 '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh',
 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf',
 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp',
 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf',
 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p',
 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians',
 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

Documentation sur les modules

```
>>> help(math.sin)
Help on built-in function sin in module math:

sin(x, /)
    Return the sine of x (measured in radians).
>>> help(math.radians)
Help on built-in function radians in module math:

radians(x, /)
    Convert angle x from degrees to radians.
>>> math.sin(math.radians(45))
0.7071067811865475
```

Remarque : pour le moment nous ignorons la signification du “, /” dans l’aide affichée par `help`, mais nous y reviendrons.

Définir de nouvelles fonctions

Pourquoi créer des fonctions ?

Diviser un programme en fonctions permet de

- **rassembler** et **donner un nom** à un groupe d'instructions :
programme plus facile à lire et à déboguer
- **éliminer du code répétitif** : programmes plus courts, plus faciles
(et moins dangereux) à modifier
- **diviser** des tâches importantes en petits morceaux : permet de
déboguer chaque morceau indépendamment
- **réutiliser** du code : des fonctions bien conçues (et souvent
silencieuses) sont souvent utiles pour d'autres programmes

Définition de nouvelles fonctions

Au chapitre précédent, nous avons vu comment créer une fonction qui s'utilise ensuite comme les fonctions prédéfinies.

```
import math

def circonference(r):
    return 2 * math.pi * r

print(circonference(10))
print(circonference(2))
```

L'exécution de ce script donnera :

```
62.831853
12.5663706
```

Définition de nouvelles fonctions

Définition

Une **définition de fonction** spécifie le nom, les arguments (optionnel) et la séquence d'instructions de la fonction.

```
def circonference(r):  
    return 2 * math.pi * r
```

```
def get_sum(x, y):  
    return x + y
```

Une définition de fonction comporte deux éléments : l'**en-tête** et le **corps**.

Définition de nouvelles fonctions : en-tête

```
def get_sum(x, y):  
    return x + y
```

Spécification de l'en-tête de la fonction :

- `def` est un mot-clef qui indique qu'il s'agit d'une définition, il est suivi par le nom de la fonction
- la liste des noms des arguments (séparés par des virgules) est donnée entre parenthèses
- les parenthèses sont obligatoires (même s'il n'y a pas d'argument)
- la liste des arguments est suivie du caractère : (deux points)
- les noms des fonctions suivent les mêmes règles que les noms des variables (cf. Chap. 2)

Définition de nouvelles fonctions : en-tête

Le nombre d'arguments

- peut être égal à zéro (mais les parenthèses restent nécessaires) ;
- n'est pas limité.

Exemple : le script suivant

```
def get_hello_word():  
    return 'Bonjour'  
  
def get_name(lastname, firstname, title):  
    return title + ' ' + firstname + ' ' + lastname  
  
print(get_hello_word(), get_name('Baroud', 'Bill', 'M.'))
```

affiche

Bonjour M. Bill Baroud

Définition de nouvelles fonctions : corps

```
def get_sum(x, y):  
    return x + y
```

Spécification du **corps** de la fonction :

- les instructions utilisent les arguments comme des variables ;
- les instructions doivent être **indentées** : cela permet de grouper les instructions (spécifier ce qui fait partie du corps). Cela fait partie de la **syntaxe** (obligatoire) ;
- en pratique, l'indentation est un nombre constant de caractères “espace” en début de chaque ligne (**convention : 4 espaces**) ;
- évitez les tabulations pour indenter (problèmes entre éditeurs) ;
- on utilise le mot-clef `return` pour retourner une valeur.

Définition de nouvelles fonctions : corps

Dans le corps d'une fonction,

- le nombre d'instructions n'est pas limité (mais respectez l'indentation) ;
- d'autres fonctions peuvent être appelées.

Exemple : le script suivant

```
def get_sum(x, y):  
    return x + y  
  
def average(a, b, c, d):  
    total = 0  
    total = get_sum(total, a)  
    total = get_sum(total, b)  
    total = get_sum(total, c)  
    total = get_sum(total, d)  
    moyenne = total / 4  
    return moyenne
```

```
print(average(2, 4, 5, 10))
```

affiche

5.25

Définition de nouvelles fonctions : corps

La lecture d'une instruction `return` termine l'exécution de la fonction. La valeur qui suit cette instruction est directement retournée et la fonction est interrompue.

Exemple : le script suivant

```
def fonction():  
    res = 3  
    return res  
    res = 5
```

```
x = fonction()  
print(x)
```

affiche

3

Définition de nouvelles fonctions

En mode interactif, il faut mettre une ligne blanche pour terminer la définition d'une fonction.

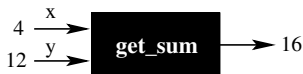
```
>>> def get_sum(x, y):  
    return x + y
```

```
>>>
```


Appel de ses propres fonctions

Une fois définie, une fonction personnelle s'appelle de la même façon que les fonctions prédéfinies. Les appels de fonctions et les expressions peuvent être **composées**.

```
>>> import math
>>> get_sum(4, 12)
16
>>> get_sum(math.pi, math.e)
5.85987448205
>>> get_sum(circonference(10), circonference(2))
75.39822368615503
```



Paramètres

```
>>> def get_sum(x, y):  
    return x + y
```

A l'intérieur d'une fonction, les arguments sont appelés des **paramètres**.

- Ils fonctionnent comme des variables (qui sont définies lors de l'appel à la fonction) ;
- La fonction marche avec n'importe quel type de paramètre sur lequel les instructions sont valides.

```
>>> get_sum('Bon', 'jour')  
Bonjour
```

Paramètres

Un argument est évalué avant l'appel à la fonction. Une variable peut être utilisée comme argument : le nom de la variable peut être différent de celui du nom du paramètre. Elle se contente de « transmettre » sa valeur au paramètre.

Illustration : visualisation du script (`arguments.py`) suivant via Python Tutor

```
import math

def circonference(r):
    return 2 * math.pi * r

def get_sum(x, y):
    return x + y

print(get_sum(3, circonference(9 - 4)))
a = 13
b = 5
print(get_sum(a, b))
```

Ordre des arguments

L'ordre des arguments est important !

```
>>> def get_ratio(x, y):  
    return x / y
```

```
>>> print(get_ratio(12, 4))
```

```
3.0
```

```
>>> print(get_ratio(4, 12))
```

```
0.333333333333
```

Variables locales

Un paramètre est une **variable locale** à sa fonction, c'est à dire qu'il n'existe pas en dehors de sa fonction (en dehors de `get_sum`, `x` et `y` n'existent pas)

```
>>> def get_sum(x, y):  
    return x + y  
  
>>> a = get_sum(3, 4)  
>>> print(a)  
7  
>>> print(x)  
NameError: name 'x' is not defined
```

Intuition : boîte noire (on ne voit pas le corps de la fonction de l'extérieur de celle-ci)

Variables locales

De la même manière, toute variable définie à l'intérieur de la fonction est une **variable locale**. Elle est détruite quand l'appel à la fonction est terminé.

```
>>> def get_sum(x, y):  
    res = x + y  
    return res  
  
>>> a = get_sum(3, 4)  
>>> print(a)  
7  
>>> print(res)  
NameError: name 'res' is not defined
```

On parle de la **portée** (scope) des variables : zone du programme dans laquelle elle est disponible (**illustration** via Python Tutor ([scope.py](#)))

Effets de bord

Effets de bord

Définition

On dit qu'une fonction possède un **effet de bord** si celle-ci produit un effet qui est visible en dehors de la fonction. **Le fait de retourner une valeur n'est pas considéré comme un effet de bord.**

Exemples d'effets de bord :

- afficher quelque chose à l'écran ;
- modifier un fichier ;
- objet passé en argument modifié après l'appel à la fonction ¹.

1. On y reviendra plus tard car pour le moment les types des arguments utilisés (int, str, etc.) ne sont pas sujets à ce type d'effets de bord.

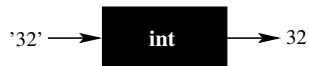
Effets de bord

Les fonctions comme `abs` ou `int` ont-elles des effets de bord ?

```
>>> result = int('32')
```

```
>>> print(result)
```

```
32
```



Effets de bord

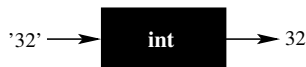
Les fonctions comme `abs` ou `int` ont-elles des effets de bord ?

Non. En effet, rien n'est affiché ou visible en dehors de la fonction. La seule chose “visible” (ou plutôt accessible) est la valeur retournée qu'il faut afficher explicitement, si nécessaire :

- soit en utilisant `print` (mode script ou interactif) ;
- soit en faisant un simple appel à la fonction en mode interactif, ce qui est raccourci déjà vu précédemment qui permet d'éviter d'entrer `print`

```
>>> result = int('32')  
>>> print(result)
```

32



Effets de bord

La fonction `math.sin` a-t-elle des effets de bord ?

```
>>> math.sin(3)  
0.1411200080598672
```

Effets de bord

La fonction `math.sin` a-t-elle des effets de bord ?

```
>>> math.sin(3)  
0.1411200080598672
```

Non. Ici, l'affichage de la valeur retournée est due à l'utilisation du mode interactif. En mode script, ou si on avait assigné la valeur de retour à une variable, rien n'aurait été affiché.

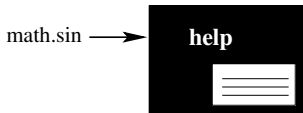
Quizz : quelles fonctions présentées jusqu'ici possèdent des effets de bord ?

Effets de bord

Les fonctions `print` et `help` provoquent un affichage. Un comportement de ces fonctions est donc visible en dehors de ce qu'elles pourraient retourner, comme le code suivant le démontre en utilisant une assignation pour éviter l'affichage automatique.

```
>>> x = print(3)
3
>>> x = help(math.sin)
Help on built-in function sin in module math:
sin(...)
    sin(x) -> Return the sine of x (measured in radians).
```

Intuition : un écran d'affichage est apparu sur la boîte noire



Effets de bord

Règle générale : les fonctions ne devraient **pas avoir d'effet de bord**, sauf si l'objet de la fonction est explicitement d'en produire un (comme `print`, `help` ou une fonction qui sauvegarderait des données dans un fichier).

Pourquoi ?

Effets de bord

Règle générale : les fonctions ne devraient **pas avoir d'effet de bord**, sauf si l'objet de la fonction est explicitement d'en produire un (comme `print`, `help` ou une fonction qui sauvegarderait des données dans un fichier).

Pourquoi ?

Car l'utilisateur d'une fonction sans effet de bord

- peut **contrôler** ce qui, dans son programme, est affiché ou pas ;
- n'a pas le risque de voir des données ou des valeurs **modifiées** sans qu'il le réalise lui même.

Imaginez si `math.sin` affichait intempestivement des choses dans un gros programme de calculs scientifiques !

Effets de bord

A contrario, certaines fonctions ont explicitement comme but d'avoir un effet de bord, comme afficher quelque chose.

```
>>> def print_jacques():  
    s = 'Frère Jacques'  
    print(s + ', ' + s)  
  
>>> print_jacques()  
Frère Jacques, Frère Jacques
```


Fonctions sans instruction `return`

```
>>> def print_jacques():  
    s = 'Frère Jacques'  
    print(s + ', ' + s)
```

La fonction précédente a été définie sans instruction `return`, car ce n'était pas son but de retourner quelque chose.

Que se passe-t-il si on essaye néanmoins d'afficher ce qu'elle retourne ?

Valeur de retour `None`

```
>>> x = print_jacques()  
Frère Jacques, Frère Jacques  
>>> print(x)  
None
```

En réalité, en Python, une fonction retourne **toujours** quelque chose mais si son but n'est pas de produire une valeur de retour, alors la valeur spéciale `None` est retournée. C'est la valeur "vide" en Python.

Remarques :

- La valeur `None` est la seule valeur possible d'un type qui lui est dédié, le `NoneType` ;
- Les fonctions `print` et `help` retournent également `None`.

return $\neq$ print

Lors du cours et des TPs, nous insisterons très fort, dans le cadre des fonctions, sur le fait que

`return  $\neq$  print`

La fonction `print` ne doit être utilisée qu'exceptionnellement dans une fonction : uniquement si le rôle de la fonction est d'afficher quelque chose (cela constituera une minorité des fonctions que vous allez définir).

Nous allons illustrer la différence entre `return` et `print` et l'intérêt des fonctions sans effets de bord en créant notre premier module.

return $\neq$ print : la bonne manière de faire

Nous créons un module `mymath.py` qui contient le code suivant.

```
def square(x):  
    return x ** 2
```

Il n'y a rien d'autre à faire pour créer un module que de créer un fichier. Nous créons également un script `test.py` qui va importer notre nouveau module.

```
import mymath  
  
x = mymath.square(5)  
print(x)
```

L'exécution du script `test.py` va afficher 25 comme attendu.

Remarque : créer un module en Python est aussi simple que ça ! Plus d'informations sur les modules : voir tutoriel "Modules" sur Moodle.

return $\neq$ print : la mauvaise manière de faire

Dans le scénario précédent, nous avons créé une fonction `square` sans effet de bord grâce à l'utilisation de `return`. C'est dans le script de test que nous avons décidé d'afficher le résultat.

Que ce serait-il passé si nous avions utilisé `print` plutôt que `return` en définissant la fonction `square` ?

Pour le tester, modifions notre module `mymath.py` avec le code suivant.

```
def square(x):  
    print(x ** 2)
```

Nous ne modifions pas le script `test.py` :

```
import mymath  
  
x = mymath.square(5)  
print(x)
```

Qu'est-ce qui va s'afficher en exécutant `test.py` ?

return $\neq$ print : la mauvaise manière de faire

Le résultat affiché est le suivant :

```
25  
None
```

C'est presque le résultat souhaité sauf qu'il y a un `None` qui s'affiche à cause du `print(x)` et du fait que comme il n'y a plus d'instructions `return` dans `square`, cette fonction retourne `None`.

On pourrait être tenté de corriger les choses en supprimant `print(x)` du script `test.py`. Et en apparence, ça fonctionne car maintenant l'affichage est simplement 25.

return $\neq$ print : la mauvaise manière de faire

Cependant, la « correction » précédente est une erreur car nous ne pouvons plus travailler avec la valeur qui est censée être calculée par la fonction `square`. Cela s'observe par exemple avec le nouveau script `test.py` suivant :

```
import mymath

x = mymath.square(5)
print(x + 5)
```

On aimerait voir 30 s'afficher mais le script produit une exception et un effet de bord, alors que cela aurait parfaitement fonctionné avec la bonne manière de faire (utiliser un `return`).

```
25
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

Flot d'exécution

Flot d'exécution

L'ordre dans lequel des instructions sont exécutées est appelé le **flot d'exécution**

- l'exécution commence toujours à la première instruction
- les instructions suivantes sont ensuite exécutées de haut en bas, une après l'autre
- une fonction doit être définie (ou importée depuis un module) **avant** d'être appelée
- les définitions de fonctions n'altèrent pas le flot d'exécution (elles sont lues et deviennent disponibles) **mais** le corps d'une fonction n'est exécuté que quand une fonction est appelée
- un appel à une fonction peut être vu comme un détour dans le programme
- le flot d'exécution n'est donc pas linéaire

Pile de fonctions

Les fonctions qui appellent d'autres fonctions forment une **pile de fonctions** : la dernière fonction appelée (disons f_B) doit être terminée pour “revenir” à la fonction f_A qui contenait l'appel à B .

Exemple : le script suivant est valide et affiche 12. Illustration de la pile via Python Tutor (`pile.py`)

```
def f_A(x):  
    return x + f_B(x)  
  
def f_B(y):  
    return y ** 2  
  
print(f_A(3))
```

Pile de fonctions

Si une exception se produit pendant l'appel à une fonction, Python affiche la pile des fonctions en cours.

Exemple : remplaçons le code de `f_B` avec une instruction erronée (`x` n'est pas accessible dans `f_B`)

```
def f_A(x):  
    return x + f_B(x)  
  
def f_B(y):  
    return x * y  
  
print(f_A(3))
```

Pile de fonctions

L'interpréteur signale l'exception et affiche la pile au moment où celle-ci s'est produite :

```
Traceback (most recent call last):  
  File "pile2.py", line 7, in <module>  
    print(f_A(3))  
  File "pile2.py", line 2, in f_A  
    return x + f_B(x)  
  File "pile2.py", line 5, in f_B  
    return x * y  
NameError: name 'x' is not defined
```

Flot d'exécution

Exercice : l'ordre des définitions est-il valide ?

```
def print_jacques():  
    print(get_twice('Frere Jacques'))  
  
def get_twice(s):  
    return s + ', ' + s  
  
def print_song():  
    print_jacques()  
    print_dormez()  
  
def print_dormez():  
    print(get_twice('Dormez-vous ?'))  
  
print_song()
```

Illustration : via
Python Tutor

Documenter son code

Commentaires

Pour être plus simple à lire, un code source devrait être correctement **documenté**.

On peut ajouter des notes dans du code, qui ne seront pas interprétées. On appelle cela des **commentaires**.

Une manière de faire en Python : tout ce qui suit le symbole `#` est ignoré jusqu'à la fin de la ligne.

```
# calcule le pourcentage de l'heure écoulée  
percent = (minute * 100) / 60
```

```
v = 5    # assigne 5 à v
```

```
v = 5    # vitesse en metres / secondes
```

Quels commentaires trouvez-vous utiles et pourquoi ?

Documenter ses fonctions

On a vu qu'on peut obtenir de l'aide avec `help`.

```
>>> help(math.sin)
Help on built-in function sin in module math:

sin(...)
    sin(x)

    Return the sine of x (measured in radians).
```

Qu'en est-il de nos fonctions ?

```
>>> help(get_sum)
Help on function get_sum in module __main__:

get_sum(x, y)
```


Documenter ses fonctions

Un **commentaire multilignes** commence par `"""` et se termine par `"""`.

```
""" this program makes
some useful computation
and is much commented.
"""
x = 2 * 3
y = x + 3 # this is a one line comment
```

Différence entre les commentaires multilignes et les commentaires “fin de ligne” : l’interpréteur ne les ignore pas toujours car ils peuvent être utilisés pour documenter certaines parties du programme, comme une fonction.

Documenter ses fonctions

Un **docstring** est un commentaire multilignes placé au début du corps d'une fonction.

```
>>> def get_sum(x, y):  
    """ return the sum of x and y """  
    return x + y
```

```
>>> help(get_sum)  
Help on function get_sum in module __main__:
```

```
get_sum(x, y)  
    return the sum of x and y
```

- Documenter clairement ses fonctions : très bonne habitude à prendre !
- Seules les instructions du corps sont cachées, toute l'information utile à l'utilisation d'une fonction est accessible.
- Formatez vos commentaires multilignes pour qu'ils soient facilement lisibles (par ex., max 79 caractères par ligne)

Documenter ses fonctions

Les **annotations de type** peuvent aussi servir à documenter une fonction, pour avertir l'utilisateur de quels sont les types attendus en entrée et en sortie.

```
def get_sum(x: int, y: int) -> int:
    """ return the sum of x and y """
    return x + y
```

Actuellement, en Python, l'annotation de type ne sert que de documentation pour le programmeur (l'interpréteur ne teste pas si une variable est du type attendu).

Introspection

Les docstring montrent une des formes de l'[introspection](#) : particularité du langage Python qui lui permet de s'auto-explorer.

```
>>> print(get_sum.__doc__)
return the sum of x and y
>>> print(get_sum.__name__)
get_sum
>>> print(math.sqrt.__module__)
math
>>> print(get_sum.__module__)
__main__
```

Le module `__main__` est le module par défaut.

Objets et introspection

Définir une fonction crée une variable avec le même nom, dont la valeur est un **objet fonction** (de type `function`)

```
>>> print(get_sum)
<function get_sum at 0x10167e048>
>>> print(type(get_sum))
<class 'function'>
```

→ c'est sur (grâce à) cet objet fonction que nous pouvons utiliser l'introspection.

Objets et introspection

On utilise la notation point pour accéder aux **attributs** d'un objet. La liste des attributs d'un objet est disponible via `dir`.

```
>>> dir(get_sum)

['__annotations__', '__call__', '__class__', '__closure__', '__code__',
 '__defaults__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__get__', '__getattr__', '__globals__',
 '__gt__', '__hash__', '__init__', '__kwdefaults__', '__le__', '__lt__',
 '__module__', '__name__', '__ne__', '__new__', '__qualname__',
 '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
 '__str__', '__subclasshook__']

>>> print(get_sum.__doc__)
return the sum of x and y
```

Illustration

Pour illustrer les fonctions et la documentation de fonctions, revenons à nos tortues perdues dans des Labyrinthes :

- lecture de la documentation de `uturtle` (et expérimentation)
- illustration avec une première version d'un module `draw`

Un petit mot sur les Tuples (parenthèse)

Tuples

En mathématiques, on représente un point dans $\mathbb{R}^n$ comme un n -uplet : collection ordonnée (séquence) de n coordonnées. Par exemple, l'origine du plan est $(0,0)$ et un point dans l'espace peut être représenté par (x,y,z) où x , y et z sont ses coordonnées dans les 3 dimensions de l'espace.

En Python, un **tuple** peut être vu et écrit de manière similaire : séquence de plusieurs valeurs séparées par des virgules et mises entre parenthèses.

```
>>> x = 2
>>> y = 3
>>> point1 = (x, y)
>>> point2 = (4, 5)
>>> type(point1)
<class 'tuple'>
>>> print(point1, point2)
(2, 3) (4, 5)
```

Tuples

On peut assigner les valeurs d'un tuple dans un autre (possédant le même nombre de valeurs)

```
>>> (a, b) = point1
>>> print(a)
2
>>> print(b)
3
```

Ceci permet de réécrire notre premier algorithme (échanger deux valeurs) très simplement et sans utiliser de variables temporaires :

```
>>> a = 17
>>> b = 21
>>> (b, a) = (a, b)
>>> print(a, b)
21 17
```

Tuples

Les valeurs d'un tuple ne doivent pas être du même type.

```
>>> matricule = 142543
>>> nom = 'John Doe'
>>> student = (nom, matricule)
>>> print(student)
('John Doe', 142543)
```

Une fonction peut prendre un tuple en paramètre.

```
>>> def sum_pair(t):
    (x, y) = t
    return x + y

>>> sum_pair(point1)
5
```

Tuples

Une fonction peut retourner un tuple, ce qui lui permet par exemple de retourner plusieurs valeurs :

```
>>> def f(x, y):  
    sum = x + y  
    prod = x * y  
    return sum, prod
```

```
>>> s, p = f(3, 4)  
>>> print(s, p)  
7 12
```

Erreurs fréquentes

Erreurs fréquentes

Indentation : problèmes quand des espaces et des tabulations sont utilisées (utilisez des espaces exclusivement)

Sauvegarde : n'oubliez pas de sauver votre script avant de l'exécuter, sinon vous ne comprendrez pas pourquoi votre programme ne fonctionne pas (même s'il est correct)

Instruction return : l'instruction `return` termine l'exécution de la fonction, la suite n'est pas exécutée

print : il ne faut pas utiliser `print` au lieu de `return` dans une fonction qui ne doit pas avoir d'effet de bord

Exercices interactifs

Une fonction est un objet que l'on peut passer en argument à une autre fonction (en omettant les parenthèses). Exemple :

```
def do_twice(f):  
    f()  
    f()
```

```
def print_spam():  
    print('spam')
```

```
do_twice(print_spam)
```

Exercice

Comprenez-vous le code ci-dessus ? Essayez-le en mode script.

Exercice

En éditant le script de l'exercice précédent :

- Modifiez la fonction `do_twice` pour qu'elle prenne deux arguments : une fonction et une valeur. Elle appelle alors la fonction deux fois, en passant la valeur en argument.
- Modifiez la fonction `print_spam` pour qu'elle affiche une chaîne passée en argument.
- Utilisez la version modifiée de `do_twice` pour appeler `print_spam` deux fois, en passant `'hello'` en argument.

Glossaire

Glossaire

fonction : séquence d'instructions qui possède un nom. Les fonctions peuvent prendre des arguments (ou pas) et peuvent retourner une valeur (ou pas : retourne `None`).

définition de fonction : instruction qui crée une nouvelle fonction, spécifie son nom, ses paramètres, et les instructions qu'elle doit exécuter.

en-tête : (header) la première ligne de la définition d'une fonction (`def`, nom de la fonction, liste d'arguments, caractère "deux points").

corps : (body) séquence d'instructions dans une définition de fonction.

paramètre : variable utilisée à l'intérieur d'une fonction qui réfère à la valeur passée en argument.

appel de fonction : instruction qui exécute une fonction. Elle consiste en le nom de la fonction suivi par une liste d'arguments entre parenthèses.

argument : valeur fournie à une fonction quand une fonction est appelée. Cette valeur est assignée au paramètre correspondant dans le corps fonction.

variable locale : variable définie à l'intérieur d'une fonction. Une variable locale ne peut être utilisée qu'à l'intérieur de sa fonction.

Glossaire

portée : la portée d'une variable est la zone du programme dans laquelle elle est disponible. La portée d'une variable locale ou d'un paramètre est limitée à sa fonction.

valeur de retour : le résultat d'une fonction. Si un appel de fonction est utilisé comme une expression, sa valeur de retour est la valeur de l'expression.

module : fichier qui contient une collection de fonctions et d'autres définitions.

import : instruction qui lit un module et crée un objet module.

__main__ : le module `__main__` est le module par défaut

composition : utiliser une expression comme une partie d'une expression plus grande, ou une instruction comme une partie d'une instruction plus grande.

flot d'exécution : ordre dans lequel les instructions sont exécutées dans un programme.

diagramme de pile : représentation graphique d'une pile de fonctions, leurs variables, et les valeurs qu'elles réfèrent.

traceback : liste de fonctions qui sont exécutées, affichées quand une exception se produit.

Glossaire

commentaire : information dans un programme destinée au lecteur du code source et qui n'a pas d'effet sur l'exécution du programme.

docstring : commentaire multilignes placé au début du corps d'une fonction, destiné à sa documentation.

introspection : particularité du langage Python qui lui permet de s'auto-explorer.

objet fonction : valeur créée par la définition d'une fonction. Le nom de la fonction est une variable qui réfère à un objet fonction.

objet module : valeur créée par une instruction `import` et qui fournit un accès aux valeurs et fonctions définies dans le module.

notation point : (dot notation) syntaxe pour appeler une fonction ou utiliser une variable définie dans un module en spécifiant le nom du module suivi d'un point, et du nom de la fonction ou de la variable. Permet également d'accéder aux attributs d'un objet fonction ou d'un objet module.

tuple : en Python, un `tuple` est une séquence de valeurs. Ils permettent par exemple de retourner plusieurs valeurs dans une fonction.